BILINGUAL ENGINEERING REFERENCE

MODERN .NET ARCHITECTURE SERIES

AI-Assisted Professional Engineering with .NET

Mastering Architectural Judgment, Clean Design, and Core System Ownership in the Era of Generative AI

AUTHORED BY

Tarek Najem

Software Architect • .NET Specialist

.NET 8/9/10
PRODUCTION CORE

القسم الأول — لماذا الـ Prompting ليس هندسة حتى تجعله كذلك

مشكلة السلسلة السحرية، مُعادةُ الصياغة

كل مهندس .NET ورث قاعدة كود تحتوي على سلسلة نصية – String Literal مُدرَجة مباشرةً في استدعاء دالة — سلسلة اتصال بقاعدة بيانات، أو رمز حالة، أو مفتاح إعداد — مُكرَّرة عبر عشرات نقاط الاستدعاء، كل نسخة منها تنحرف قليلاً عن الأخرى مع مرور الوقت. الحل معروف ومتبع منذ عقود: استخراج القيمة إلى ثابت مُسمَّى، أو كائن إعداد مُحدَّد النوع، أو عميل مُولَّد، بحيث يكون هناك مكان واحد تُعرَّف فيه القيمة وكل مُستهلِك يُشير إلى ذلك المكان الواحد بدلاً من إعادة كتابتها.

الطريقة التي تتعامل بها معظم الفرق الهندسية حالياً مع النماذج اللغوية الكبيرة تُعيد إنتاج هذه المشكلة بالضبط، على نطاق أوسع وبمخاطر أعلى. الموجّه – Prompt — تعليمات النظام، والإطار، والقيود، وطلب التنسيق — يُكتب طازجاً في واجهة محادثة أو يُبنى مباشرةً داخل كود التطبيق، يُستخدَم مرة واحدة، ثم إما يُهمَل أو يُتذكَّر جزئياً في المرة التالية التي تظهر فيها مهمة مشابهة. مهندسان يحلّان مشكلتين متجاورتين مقابل النموذج ذاته يكتبان موجّهين مختلفين ينتجان نتائج متشابهة سطحياً اليوم، ويتباعدان بصمت مع تحديث النموذج الأساسي، أو مع توسّع الحالات الحدية للمهمة. لا شيء يمنع هذا التباعد لأن لا شيء يُعامل الموجّه كقطعة أثر لها هوية وإصدار وعقد.

يُسمّي هذا القسم المشكلة صراحةً ويُرسي البديل: يُصبح الموجّه هندسةً، بنفس المعنى الذي تُصبح فيه السلسلة السحرية نظام إعداد، حين يُستخرَج ويُصدَّر ويُحدَّد نوعه عند حدوده ويُعاد استخدامه عمداً بدلاً من إعادة كتابته من الذاكرة.

ما يعنيه "الـ Prompting" عادةً ولماذا هذا غير كافٍ

دخل مصطلح "هندسة الموجّهات – Prompt Engineering" الاستخدام الشائع لوصف ممارسة صياغة الطلبات لنموذج لغوي بطرق تُنتج نتائج أفضل. هذه الممارسة حقيقية والمهارة مهمة. لكن المصطلح يخلط بين نشاطين يستحقان الفصل لأغراض هندسية: حِرفة صياغة تعليمة فعّالة، وانضباط جعل تلك التعليمة مكوناً موثوقاً وقابلاً للمراجعة وقابلاً لإعادة الاستخدام في نظام برمجي.

النشاط الأول — الصياغة — شيء يُحسَّن بالممارسة ولا يُنتج بمفرده قِطعاً هندسية. موجّه مصاغ جيداً كُتب مرة واحدة في نافذة محادثة وأنتج نتيجة مفيدة لا يختلف بشكل ذي معنى عن تعليمة شفهية مصاغة جيداً لكن غير موثَّقة: نجحت هذه المرة، لهذا الشخص، في هذا السياق، ولا توجد قطعة أثر تحفظ لماذا نجحت.

النشاط الثاني — معاملة الموجّهات كهندسة — يتطلب أن يوجد الموجّه كقطعة أثر مُصدَّرة بمدخلات صريحة وقيود صريحة وعقد صريح لما يُحسَب خرجاً صالحاً. هذه فئة مختلفة كلياً من الممارسة، تُشابه الفارق بين مطوّر يكتب استعلام SQL يدوياً كل مرة وفريق يُعرّف إجراءً مُخزَّناً – Stored Procedure مُعَلْمَناً يستدعيه كل مُستدعٍ بشكل متطابق.



شكل ١: Diagram: dg-23729e179b0a

لماذا آليات الفصل الثاني تجعل هذا غير اختياري

أرسى القسم الأول من الفصل الثاني أن النموذج لا يملك ذاكرة مستمرة بين الاستدعاءات فيما يتجاوز ما أُدرج صراحةً في نافذة السياق — كل فارق دقيق في القصد يجب إعادة ترميزه، بالكامل، في كل موجّه على حدة. أرسى القسم الثاني أن القدرة والموثوقية تتباينان حسب كثافة التوزيع التدريبي، مما يعني أن صياغة موجّه نجحت لشكل مهمة واحد قد تتراجع أداءً بصمت لشكل مهمة مجاور. أرسى القسم الرابع تصنيف الهلوسة، وأرسى القسم الخامس نمط الحد الحتمي، وكلاهما يفترض مسبقاً وجود عقد مُعرَّف لما يبدو عليه الخرج "الصالح" — افتراض لا يُلبّيه الـ Prompting العشوائي.

بصراحة: كل آلية أرساها هذا الكتاب حتى الآن حول كيفية تصرف النماذج اللغوية هي آلية غير مُجهَّزة بنيوياً لإدارتها الـ Prompting العشوائي غير المُصدَّر بأمان.

نمط عقد الموجّه

```
// Target Framework: .NET 10.0
// Chapter: 03 | Section: 01
// book/chapters/chapter-03/sections/section-01.ar.md
//
// PromptContract<TInput, TOutput>: موجّه مُمثَّل كقطعة أثر .NET
// مُصدَّرة ومُحدَّدة النوع بدلاً من سلسلة نصية مضمَّنة.

namespace EngineeringCommunicationToolkit.Core.Prompts;

public abstract class PromptContract<TInput, TOutput>
{
    public abstract string ContractId { get; }
    public abstract string Version    { get; }
    public abstract string RenderSystemPrompt();
    public abstract string RenderUserPrompt(TInput input);
    public string FullyQualifiedId => $"{ContractId}@{Version}";
}

public sealed class CodeReviewPromptContract
    : PromptContract<CodeReviewRequest, string>
{
    public override string ContractId => "code-review-comment";
    public override string Version    => "1.2.0";

    private const string CodeFence = "\textbackslash{}u0060\textbackslash{}u0060\textbackslash{}u0060";

    public override string RenderSystemPrompt() =>
```

```
        "You are a senior .NET engineer performing a code review. " +
        "Identify concrete issues only. For each issue, cite the specific " +
        "line or pattern and explain the concrete risk.";

    public override string RenderUserPrompt(CodeReviewRequest input) =>
        $"Review the following {input.Language} code from {input.FileName}:\textbackslash{}n" +
        $"{CodeFence}{input.Language}\textbackslash{}n{input.Code}\textbackslash{}n{CodeFence}\textbackslash{}n" +
        $"Focus areas: {string.Join(", ", input.FocusAreas)}";
}

public sealed record CodeReviewRequest(
    string FileName,
    string Language,
    string Code,
    IReadOnlyList<string> FocusAreas);
```

خاصيتان في هذا النمط تستحقان التصريح. أولاً، `RenderSystemPrompt` و`RenderUserPrompt` دالتا .NET حتميتان — قابلتان للاختبار بالكامل بتأكيدات تقليدية، مُطابِقتَان تماماً لإطار الفصل الثاني القسم الخامس للحد الحتمي. ثانياً، `Version` ليست زخرفية — تُعطي الفريق آلية ملموسة للإجابة عن "هل تغيّر هذا الموجّه؟" أثناء التحقيق في حادثة.

الجانب الآخر من الميزان: التكلفة

معاملة الموجّهات كقِطع هندسية ليست مجانية. تعريف فئة فرعية من `PromptContract`، واتخاذ قرار بشأن سياسة الإصدارات، وصيانة منطق تحويلها كمكوّن مُختبَر ومُراجَع، يستغرق وقتاً هندسياً حقيقياً. للتفاعل الاستكشافي لمرة واحدة — سكربت يُشغَّل مرة واحدة للتحقيق في حادثة منفردة — هذا العبء غير مُبرَّر.

القرار الذي يحتاج الفريق إلى اتخاذه هو أين تقع نقطة التعادل في سياقه المحدد. موجّه يُستدعى من ثلاث نقاط استدعاء بعمر متوقع ستة أشهر يُبرّر عبء النمط براحة. موجّه يُستدعى مرة واحدة، من مهندس واحد، لمهمة لن تتكرر، لا يُبرّره. الفرق التي تتعامل مع هذا الانضباط للتوّ تُخطئ غالباً في أحد اتجاهين متوقعين: إضفاء الطابع الرسمي على كل تفاعل بصرف النظر عن إمكانية إعادة استخدامه، أو عدم إضفاء الطابع الرسمي على أي شيء. لا يعكس أي من الطرفين الحكم الذي يطلبه هذا القسم — الهدف هو مطابقة مستوى الانضباط الهندسي مع مستوى إعادة الاستخدام والعواقب التي يحملها الموجّه المحدد فعلياً، وهو الحكم النسبي ذاته الذي يُطبّقه مهندسو .NET بالفعل حين يقرّرون ما إذا كان جزء من المنطق يستحق الاستخراج إلى مكتبة مشتركة.

ملاحظة حول انضباط التسمية

قيم `ContractId` يجب أن تتبع نفس انضباط التسمية الذي يُطبّقه الفريق بالفعل على مسارات API أو أسماء مواضيع ناقل الرسائل – Message Bus — وصفية، وثابتة، ومُصنَّفة حسب الاهتمام لا حسب الفريق أو المؤلف الفردي. عقد مُسمَّى `code-review-comment` يُوصِل غرضه لأي مهندس يصادفه بعد أشهر؛ عقد مُسمَّى باسم الفرد الذي كتبه أو برقم تذكرة أُغلقت

منذ زمن طويل لا يفعل ذلك. هذا انضباط صغير لكنه يتراكم قيمةً مع نمو سجل العقود، تاركاً الفريق في مكان أفضل بكثير مما كان سيكون عليه لو أُهمل الانضباط في مراحل النمو المبكرة.

استراتيجية الإصدارات وانضباط الترحيل

فئة فرعية من `PromptContract` تعيش في نظام إدارة الكود ترث انضباط الإصدارات من الآليات ذاتها التي يُطبّقها مهندسو .NET بالفعل على عقود الواجهات البرمجية وواجهات المكتبات. الإصدار الدلالي – Semantic Versioning يُقابِلها مباشرةً: زيادات التصحيح لتحسينات الصياغة التي لا تُغيّر شكل الخرج المتوقع، وزيادات ثانوية لحقول أو قيود خرج جديدة يمكن للمُستدعِين تجاهلها بأمان، وزيادات رئيسية للتغييرات البنيوية في `TInput` أو `TOutput` التي تتطلب تحديث المُستدعِين. خاصية `Version` تجعل هذه الزيادات مرئية وقت التشغيل، وخاصية `FullyQualifiedId` — التي تجمع `ContractId` و`Version` — تسمح للفريق بتسجيل الإصدار الذي أنتج خرجاً معيناً بدقة أثناء التحقيق في حادثة.

الترحيل عبر الإصدارات الرئيسية يتبع النمط ذاته الذي يستخدمه الفريق لإيقاف نقطة نهاية API: يبقى الإصدار القديم مُسجَّلاً وقابلاً للاستدعاء بينما يُهاجِر المُستدعُون فرادى، مع تسجيل إشارة إيقاف في كل مرة يُستدعى فيها الإصدار القديم. التنفيذ الملموس يُعلّق هذه الإشارة على قيد سجل التدقيق الخاص بـ `DeterministicAiBoundary<T>` من الفصل الثاني القسم الخامس، فيجعل ظهور الإيقاف تلقائياً بدلاً من اعتماده على تذكّر أحدهم للتحقق. لا يُزال الإصدار القديم إلا بعد أن يؤكد الفريق — عبر بيانات استدعاء مُسجَّلة، لا افتراض — أن لا نقطة استدعاء متبقية تعتمد عليه.

انضباط الترحيل هذا ليس عبئاً نظرياً. الفرق التي تتخطاه وتكتفي بـ "تحديث الموجّه" تكتشف التكلفة حين يتطلب التحقيق في حادثة معرفة إصدار الموجّه النشط وقت فشل محدد. بدون تتبّع الإصدارات، يتوقف التحقيق عند "تغيّر الموجّه في وقت ما من السبرنت الماضي". مع تتبّع الإصدارات، يمضي التحقيق إلى "كان الإصدار 1.2.0 من `code-review-comment` هو النشط، والتغيير من 1.1.0 أزال التعليمة الصريحة لذكر أرقام الأسطر، وهذا يفسّر تحوّل صيغة الخرج". هذا التحقيق الدقيق الواعي بالإصدار لا يكون ممكناً إلا حين يكون الإصدار خاصية من الدرجة الأولى في العقد، لا تعليقاً في وصف طلب سحب قد يكون محدَّثاً أو لا يكون.

من السلاسل السحرية إلى العقود المُحدَّدة النوع: التشابه الكامل

تشبيه السلسلة السحرية الذي يفتتح هذا القسم يمتد إلى أبعد مما يبدو للوهلة الأولى، وتتبّعه إلى نتيجته المنطقية يُوضّح النطاق الكامل لما يعنيه معاملة الموجّهات كقِطع هندسية. حين يستخرج فريق .NET قيمة إعداد من سلسلة مُضمَّنة إلى كائن `IOptions<T>` مُحدَّد النوع، فالفائدة ليست فقط أن القيمة تعيش في مكان واحد — بل أنها أصبحت قابلة للملاحظة والحقن والاختبار، وتحكم حداً يستطيع نظام الأنواع التفكير فيه. المُستدعِي لا يحتاج إلى معرفة التمثيل الداخلي للإعداد؛ إنه يعتمد على العقد المُحدَّد النوع، وشكل العقد هو الشيء الوحيد الذي يحتاج إلى البقاء مستقراً كي يبقى المُستدعُون غير متأثرين بالتغييرات الداخلية.

يُوفّر `PromptContract<TInput, TOutput>` المجموعة ذاتها من الخصائص عند الحد بين مُستدعٍ .NET وتفاعل مع الذكاء الاصطناعي. `TInput` هو العقد لما يجب أن يُوفّره المُستدعِي — ليس وصفاً حر الصياغة لـ "ما قد ترغب في تضمينه"، بل سجل مُحدَّد النوع بحقول مُسمَّاة وإلزامية. `TOutput` هو العقد لما يستقبله المُستدعِي — ليس نصاً خاماً قد تتباين صيغته حسبما صادف أن صيغ الموجّه في يوم معين، بل نوع يمكن لكود .NET المحيط استهلاكه كما يستهلِك أي نتيجة أخرى مُحدَّدة النوع. `Version` هو عقد إدارة التغيير — الانضباط ذاته الذي يُطبّقه الفريق على زيادة إصدار API، مرفوعاً إلى مستوى الموجّه بدلاً من نقطة نهاية HTTP.

هذا التأطير يجعل صريحاً لماذا لحجة القسم الأول آثار تتجاوز الموجّهات نفسها بكثير: إنها حجة حول أين ينبغي رسم الحد بين السلوك الحتمي والسلوك الاحتمالي في نظام .NET، وكيف ينبغي أن يبدو ذلك الحد. المكوّن الاحتمالي — استدلال النموذج الفعلي — ينتمي إلى داخل تنفيذ العقد، حيث يُعزَل، وحيث يُتحقَّق من مخرجاته قبل عبور الحد، وحيث تُحكَم التغييرات عليه بمعرّف الإصدار بدلاً من السماح لها بالتأثير بصمت على كل مُستدعٍ لحظة تعديل موجّه. كل ما يقع خارج ذلك الحد حتمي وقابل للاختبار وقابل للصيانة مثل أي كود .NET آخر — وهي بالضبط النتيجة الهندسية التي وُجد هذا الفصل لتأسيسها.

الربط بنمط الحد الحتمي في الفصل الثاني

يستحق الأمر إيضاح رابط واحد صريح قبل أن يمضي الفصل قدماً، لأنه يربط نمط العقد في هذا القسم بمعمارية الفصل الثاني بمستوى دقة يجعل كلا الفصلين يُعزّزان بعضهما. `DeterministicAiBoundary<T>` من الفصل الثاني القسم الخامس هو الحد بين استدعاء النموذج الاحتمالي وكود .NET الحتمي المحيط به. `PromptContract<TInput, TOutput>` من هذا القسم هو ما يسكن على الجانب الحتمي من ذلك الحد — المكوّن المسؤول عن تحضير المدخل للجانب الاحتمالي بطريقة مُصدَّرة ومُحدَّدة النوع وقابلة للاختبار باستقلالية.

معاً، يُشكّلان حداً متكاملاً وواضح التعريف: عقد داخلاً، استدعاء نموذج وتحقق في الداخل، نتيجة مُحدَّدة النوع خارجاً. كل نمط آخر يُقدّمه هذا الفصل — هندسة السياق، وسير العمل القابلة لإعادة الاستخدام، والمحادثات المُرتكِزة على المواصفات، وتقليل التباين، وإدارة الحالة عبر الخطوات — هو توسّع لأحد نصفي هذا الهيكل الأساسي.

النتيجة المؤسسية الأوسع

الحجة التي يُقدّمها هذا القسم تمتد إلى ما هو أبعد من جودة الموجّه الفردية — إنها في جوهرها حجة حول إدارة المعرفة والتعلم المؤسسي. ثقافة الـ Prompting العشوائي هي ثقافة تتراكم فيها التحسينات في أذهان الأفراد وتضيع حين يغادرون، وتُرتكَب فيها الأخطاء ذاتها بشكل متكرر من قِبَل مهندسين مختلفين يواجهون الحالة الحدية ذاتها باستقلالية، وليس للخبرة الجماعية للفريق مكان دائم. ثقافة عقد الموجّه هي ثقافة تتراكم فيها التحسينات في قِطع أثر مُصدَّرة تصمد أمام التغيير الفردي، وتُعالَج فيها الحالة الحدية المُكتشَفة من قِبَل مهندس واحد بتحديث عقد يُفيد كل نقاط الاستدعاء، وتُرمَّز خبرة الفريق الجماعية في سجل العقود لا في الذاكرة والمعرفة القبلية غير الموثَّقة.

هذا لا يختلف نوعاً عن الفائدة المؤسسية لسائر الانضباطات الهندسية — الاختبارات التي تُوثّق السلوك المتوقع، والواجهات المُحدَّدة النوع التي تُلخّص ما تقبله الخدمة وما تُعيده، وسجلات قرارات المعمارية المكتوبة التي تُلخّص لماذا اتُّخذ قرار تصميمي. إنه الانضباط ذاته مُطبَّق على فئة جديدة من مكوّنات النظام. الموجّهات التي تحكم كيفية تفاعل التطبيق مع نماذج الذكاء الاصطناعي جزء من استثماره الهندسي بنفس القدر الذي يُشكّله الكود المحيط بها. معاملتها بنفس الصرامة التي يحظى بها الكود ليست عبئاً إضافياً يُوضَع فوق الهندسة العادية — إنها الهندسة العادية مُطبَّقة باتساق على كل ما يُحدّد كيفية تصرف النظام.

عقود الموجّهات كتوثيق حيّ

فائدة نمط `PromptContract<TInput, TOutput>` تظهر بمرور الوقت في دوره كتوثيق حيّ للقدرات التي يعتمد عليها التطبيق. `CodeReviewPromptContract.ContractId` يُوثّق أن التطبيق يمتلك قدرة مراجعة كود؛ `Version` يُوثّق آخر تغيير جوهري؛ `TInput = CodeReviewRequest` يُوثّق ما تتطلبه القدرة؛ `TOutput = string` يُوثّق ما تُنتجه.

هذا التوثيق محدَّث تلقائياً بطريقة لا تكون عليها ملفات التوثيق المنفصلة، لأن فئة العقد هي تعريف القدرة ذاته لا وصف مكتوب عنه في مكان آخر. يُوائم هذا انضباط عقد الموجّه مع المبدأ الأشمل القائل بأن الكود هو التوثيق الموثوق لسلوك النظام — مبدأ يُطبّقه مهندسو .NET على الواجهات المُحدَّدة النوع، ومخططات البيانات، وعقود الخدمات. تطبيقه باتساق على عقود موجّهات الذكاء الاصطناعي يُكمل تغطية هذا المبدأ عبر كامل سطح سلوك تطبيق .NET الحديث، بما في ذلك الأجزاء التي يتوسطها الذكاء الاصطناعي.

ما الذي يُغيّره هذا الفصل في سلوك الفريق اليومي

يستحق السؤال العملي إجابةً مباشرة: ما الذي يتغير فعلياً في العمل اليومي لفريق .NET حين يتبنّى نمط عقد الموجّه؟ ثلاثة تغييرات ملموسة تحدث بشكل متسق في الفرق التي تتبنّى هذا النمط.

الأول هو أن مراجعة الكود – Code Review تُصبح مكاناً شرعياً لمناقشة تعريف الموجّه. قبل نمط العقد، الموجّهات المُدرَجة مباشرةً في كود التطبيق كانت أحياناً تمرّ في مراجعة الكود كتفاصيل تنفيذ صغيرة لا تستحق اهتماماً جوهرياً. بعد نمط العقد، `PromptContract.RenderSystemPrompt()` قطعة كود مثل أي قطعة أخرى تستحق المراجعة بنفس الصرامة التي يُراجَع بها أي منطق أعمال — المُراجِع يستطيع الاستفسار عن سبب اختيار هذه القيود بالذات، وما الفرق بين هذه الصياغة والصياغة البديلة، ولماذا هذا الإصدار بالذات وليس الإصدار السابق.

الثاني هو أن تتبع رصيد التغيير يُصبح ممكناً. حين تتصرف خاصية مدعومة بالذكاء الاصطناعي بشكل مختلف بعد نشر – Deployment، السؤال "هل تغيّر الموجّه؟" أصبح له إجابة محددة: قارن `Version` في الإصدار الجديد بـ `Version` في الإصدار القديم. إذا تغيّر الإصدار، الموجّه تغيّر، وسجل الكود يُظهر بالضبط ماذا تغيّر ومتى ومن غيّره. إذا لم يتغير الإصدار، الموجّه لم يتغير وعلى الفريق البحث في مكان آخر عن سبب التغيير في السلوك.

الثالث هو أن القدرات المدعومة بالذكاء الاصطناعي تُصبح قابلة للاكتشاف. مع مرور الوقت، يُصبح سجل عقود الموجّهات — قائمة `ContractId` ذات المعنى وإصداراتها وأنواع مدخلاتها ومخرجاتها — جرداً يمكن استعراضه لكل ما يستطيعه التطبيق بمساعدة الذكاء الاصطناعي. مهندس جديد ينضم للفريق يمكنه استعراض سجل العقود وفهم القدرات القائمة بدلاً من الاعتماد على المعرفة القبلية لمعرفة أي قدرات الذكاء الاصطناعي موجودة بالفعل.

هذه التغييرات الثلاثة تُشكّل معاً الفارق بين فريق يُعامل الذكاء الاصطناعي كصندوق أسود يُستشار بشكل مخصوص وفريق يُعامله كمكوّن من مكوّنات نظامه قابل للمراجعة والتدقيق والتطوير المُنضبَط. ذلك الفارق يتراكم قيمةً مع مرور الوقت بشكل غير خطي: في الأشهر الأولى يبدو هامشياً، وبعد عام في مشروع ناضج يكون الفارق بين نظام يمكن الاعتماد عليه وصيانته ونظام تراكمت فيه ديون الموجّهات إلى الحد الذي يُهدّد موثوقيته.

تعريف العقد قبل الاستدعاء: أثر الانعكاس على التفكير

تجربة عملية يصفها المهندسون الذين يتبنّون نمط العقد لأول مرة: فعل كتابة `RenderSystemPrompt()` و`RenderUserPrompt()` كدالتين منفصلتين مُحدَّدتَي التوقيع، قبل أي استدعاء فعلي للنموذج، يُجبر على صياغة ما يُريده المرء من التفاعل بدقة أكبر مما يحدث حين يُكتَب الموجّه مباشرةً في واجهة المحادثة. الانفصال بين مرحلة التعريف ومرحلة الاستدعاء يُشبه ما يُجبر عليه تصميم الاختبار قبل الكود في التطوير القائم على الاختبارات – Test-Driven Development: فعل التعريف يكشف افتراضات لم تكن ظاهرة حين كانت محتجزة في الذهن غير مُصاغة.

مهندس يجد صعوبةً في تعريف `RenderSystemPrompt()` بوضوح لعقد محدد على الأرجح اكتشف غموضاً حقيقياً في المهمة ذاتها — علامة على أن المهمة تحتاج مزيداً من التحليل قبل أن يبدأ أي استدعاء للنموذج، لا أن النمط مُقيِّد بشكل مُبالَغ فيه. هذه القيمة — كشف الغموض مبكراً وبتكلفة منخفضة — موجودة حتى للمهندس الذي يتبنّى انضباط الصياغة المسبقة دون البنية الرسمية الكاملة لفئة `PromptContract`. البنية الرسمية تُرسّخ هذه القيمة وتجعلها منهجية وقابلة للقياس وقابلة للمشاركة عبر الفريق، لكن القيمة الجوهرية تتحقق من انضباط التفكير نفسه.

إعداد بقية الفصل

أرسى القسم الأول الأساس: الموجّه المُعامَل كقطعة أثر مُحدَّدة النوع ومُصدَّرة وقابلة للاختبار باستقلالية هو اللبنة الأساسية للتعاون المنضبط مع الذكاء الاصطناعي. الأقسام الخمسة التالية تبني هياكل أكثر قدرةً تدريجياً على هذا الأساس. القسم الثاني يُعالج النصف المتغير من كل موجّه — السياق — ويُقدّم انضباط الميزانية والترتيب الصريح الذي تجعله ضرورياً استنتاجات الفصل الثاني حول "الضياع في المنتصف". القسم الثالث يُغلّف العقد والسياق في سير عمل مُسجَّلة وقابلة لإعادة الاستخدام. القسم الرابع يمتد بهذا الانضباط إلى المحادثات متعددة الدورات. القسم الخامس يُقدّم تقنيات مستوى التوليد التي تُضيّق تباين الخرج داخل التفاعل الفردي. القسم السادس يُغلق الفصل بمعالجة إدارة الحالة الصريحة للمهام التي تمتد عبر تفاعلات متعددة، مع نماذج التعاون من الفصل الأول مُطبَّقة كخصائص مُتتبَّعة ومُحدَّدة النوع بدلاً من اتفاقيات غير رسمية.

خاتمة القسم الأول

يبقى القسم الأول لهذا الفصل في نقطة تقاطع محورية: إنه يُحوّل الموجّه من ظاهرة مؤقتة تُستهلَك وتُنسى إلى قطعة أثر هندسية دائمة تحمل هوية وتاريخاً وعقداً. هذا التحويل ليس مجرد تحسين جمالي في تنظيم الكود — إنه تحوّل في كيفية تفكير الفريق في علاقته بالذكاء الاصطناعي: لا كأداة يُستشار عشوائياً، بل كتبعية – Dependency تُدار بنفس الانضباط الذي تُدار به أي تبعية خارجية في نظام .NET الإنتاجي. ما يليه من أقسام يبني على هذا التحوّل بإضافة الانضباط ذاته إلى السياق والسير والمحادثات متعددة الأدوار وتقليل التباين وإدارة الحالة — طبقةً فوق طبقة، كلها مبنية على الأساس المرسَّخ هنا.

الانضباط المؤسسي المتراكم عبر سجل العقود يُحوّل ما كان مجرد إلهام عابر وحرفة فردية إلى إطار هندسي مُشترَك وقابل للتطور والمراجعة. التعاون الناجح مع نماذج الذكاء الاصطناعي يبدأ دائماً بهذا القرار الهندسي الجوهري.

أرسى القسم الأول أن الموجّه لا يصبح قطعة أثر هندسية إلا حين يُستخرَج من السلاسل المضمَّنة إلى عقد مُحدَّد النوع ومُصدَّر وقابل للاختبار باستقلالية. القسم الثاني يفحص النصف المتغير من كل موجّه — السياق — وانضباط الميزانية والترتيب الصريح الذي تجعله ضرورياً استنتاجات الفصل الثاني حول "الضياع في المنتصف".

Reclaim Your Architectural Ownership

Software engineering is entering its most disruptive decade. AI tools can generate code at machine speed — but they often introduce hidden structural vulnerabilities. True engineering is not about typing speed; it is the discipline of making high-stakes decisions under uncertainty.

This technical reference bridges the gap between raw artificial intelligence capabilities and the strict architectural requirements of the .NET ecosystem. Learn to treat language models as professional collaborators without surrendering your system boundaries, type safety, or professional responsibility.

- ✓ **Specification-Driven Development:** Enforce strict design boundaries via deterministic AI workflows.
- ✓ **Enterprise Patterns:** Clean Architecture and Domain-Driven Design with C# 14 and .NET 8/9.
- ✓ **Production Guardrails:** Autonomous multi-agent systems with observability, logging, and feedback loops.
- ✓ **Bilingual Reference:** English text with full Arabic localization and right-to-left support.

About the Author
Tarek Najem is a senior software engineer and architect with extensive experience across the entire Microsoft .NET ecosystem — from its earliest releases to the latest modern frameworks. He specializes in enterprise application architecture, clean design patterns, and AI-assisted engineering workflows. Tarek is highly skilled in modernizing and refactoring legacy systems, whether using artificial intelligence or traditional engineering techniques, to meet contemporary business and technical requirements. His work focuses on bridging the gap between generative AI capabilities and professional software engineering standards.
Shelving: Software Engineering / .NET Architecture — Open Source MIT License
github.com/TarekNajem04/AI-Assisted-Professional-Engineering-with-.NET

OPEN SOURCE EDITION

v1.0 — July 2026